STUDY GUIDE

Cheat Sheets

Quick-reference sheets for Java syntax, collections, complexity and the archive algorithms

Print these or keep them beside your editor. Every entry is verified against the lessons.

Generated 10 October 2026 from the same lesson data that powers the website.
Every Java program in this guide was compiled with --release 17 and run against its expected output.

Cheat sheets

Java syntax quick sheet

Program skeleton	Meaning
`public class Main { … }`	A class named like the file (Main.java).
`public static void main(String[] args)`	Where execution starts.
`System.out.println(x);`	Print x and a newline. print = no newline; printf = formatted.
`// comment /* block */`	Ignored by the compiler.

Primitive types	Meaning
`byte / short / int / long`	8 / 16 / 32 / 64-bit integers. int max = 2 147 483 647.
`float / double`	32 / 64-bit decimals. Use double by default.
`char`	One UTF-16 character, written 'A'. Behaves like a small int.
`boolean`	true or false. if() needs a boolean — no int.
`long x = 5_000_000_000L;`	Suffix L for long literals; f for float.

Input / output	Meaning
`Scanner sc = new Scanner(System.in);`	import java.util.*;
`sc.nextInt() / nextLong() / nextDouble()`	Read one token of that type.
`sc.next() / sc.nextLine()`	One word / the rest of the line.
`System.out.printf("%d %5.2f %s%n", a, b, c);`	%d int, %f decimal, %s string, %n newline.

Control flow	Meaning
`if (c) {…} else if (c2) {…} else {…}`	Condition must be boolean.
`for (int i = 0; i < n; i++) {…}`	Counting loop.
`for (int x : arr) {…}`	for-each over arrays/collections.
`while (c) {…} do {…} while (c);`	do-while runs at least once.
`switch (v) { case 1: …; break; default: … }`	Without break execution falls through.

Operators & precedence

From highest to lowest precedence	Meaning
`() [] .`	grouping, array index, member access
`x++ x-- ++x --x +x -x ! ~`	unary
`* / %`	multiplicative (left to right)
`+ -`	additive (+ also joins strings)

From highest to lowest precedence	Meaning
`<< >> >>>`	shifts
`< <= > >= instanceof`	relational
`== !=`	equality
`& ^ |`	bitwise AND, XOR, OR (in that order)
`&& ||`	logical AND, OR (short-circuit)
`?:`	ternary (right to left)
`= += -= *= /= %= <<= >>= &= |= ^=`	assignment (right to left)

Gotchas	Meaning
`7 / 2 → 3 7 / 2.0 → 3.5`	int ÷ int is integer division.
`-7 % 3 → -1`	Remainder keeps the sign of the dividend.
`"" + 3 + 4 → "34" 3 + 4 + "" → "7"`	Left to right: numbers add until a String appears.
`a == b on Strings`	Compares references. Use a.equals(b).

Bit-manipulation toolkit

Operators	Meaning
`x & y`	AND — 1 only where both are 1
`x | y`	OR — 1 where either is 1
`x ^ y`	XOR — 1 where they differ
`~x`	NOT — flips every bit (−x−1)
`x << k`	multiply by 2^k
`x >> k`	arithmetic shift: divide by 2^k, keeps sign
`x >>> k`	logical shift: shifts in zeros

Patterns	Meaning
`(x >> i) & 1`	read bit i
`x | (1 << i)`	set bit i
`x & ~(1 << i)`	clear bit i
`x ^ (1 << i)`	toggle bit i
`x & 1`	is x odd?
`x & (x - 1)`	clear the lowest set bit
`x & -x`	isolate the lowest set bit
`(1 << n) - 1`	mask of n ones
`((b & 0xF0) >>> 4) | ((b & 0x0F) << 4)`	swap the nibbles of a byte

Patterns	Meaning
`Integer.toBinaryString(x)` `Integer.bitCount(x)`	built-in helpers

Complexity cheat sheet

Growth classes (best → worst)	Meaning
`O(1)`	array access, odd/even
`O(log n)`	binary search, Euclid, halving loops
`O(√n)`	trial division, loops while i·i ≤ n
`O(n)`	one pass, linear search
`O(n log n)`	merge sort, quick sort (average), sorting + scan
$O(n^2)$	two nested loops, bubble/selection/insertion sort
$O(n^3)$	three nested loops, naive matrix multiply
$O(2^n)$	all subsets, naive Fibonacci
`O(n!)`	all permutations

Counting rules	Meaning
`sequential blocks`	add the costs → keep the largest
`nested loops`	multiply the iteration counts
`i *= 2 or n /= 2`	$\log_2 n$ iterations
`j += i inside i-loop`	harmonic series → n log n
`T(n) = T(n-1) + O(1)`	O(n)
`T(n) = 2T(n/2) + O(n)`	O(n log n)
`T(n) = 3T(n/2) + O(n)`	O(n^1.585) — Karatsuba
`recursion space`	depth of the call stack

Java Collections quick sheet

Which one?	Meaning
`ArrayList<T>`	resizable array. get(i) O(1), add at end O(1) amortized, insert middle O(n)
`LinkedList<T>`	doubly linked list; also a Deque. No fast get(i).
`HashSet<T>`	unique items, O(1) contains/add average, no order
`HashMap<K,V>`	key → value, O(1) average. getOrDefault, merge, entrySet
`TreeMap / TreeSet`	sorted, O(log n)
`ArrayDeque<T>`	stack AND queue: push/pop/offer/poll
`PriorityQueue<T>`	min-heap by default: poll() returns the smallest

Which one?	Meaning
`Arrays.sort(a) Collections.sort(list)`	sort; comparator for custom order

String & StringBuilder

Methods you will use constantly	Meaning
`s.length() s.charAt(i)`	size / character at index i
`s.substring(a, b)`	chars a … b−1
`s.indexOf(x) s.contains(x)`	position (−1 if absent) / test
`s.equals(t) s.compareTo(t)`	content equality / ordering
`s.toCharArray() new String(arr)`	to and from char[]
`s.split(" ") String.join(",", list)`	split / join
`new StringBuilder(s).reverse().toString()`	reverse a string
`sb.append(x) sb.insert(0, x) sb.setCharAt(i, c)`	mutable edits
`c - 'a'`	index 0..25 of a lowercase letter

Formula sheet (archive topics)

Number theory	Meaning
`gcd(a, b) = gcd(b, a mod b)`	Euclid; gcd(a, 0) = a
`lcm(a, b) = a / gcd(a, b) · b`	divide first to avoid overflow
`φ(n) = n · Π(1 − 1/p)`	p over distinct primes dividing n
`C(n, r) = n! / (r!(n−r)!)`	C(n, r) = C(n−1, r−1) + C(n−1, r)
`x = Σ r_i·M_i·inv(M_i) mod M`	Chinese remainder theorem
`Josephus: J(1) = 0; J(n) = (J(n−1) + k) mod n`	0-based; add 1 for 1-based
`Equilibrium: leftSum == total − leftSum − a[i]`	single pass
`Alice's apples: 2n(n+1)(2n+1); perimeter 8n`	garden of half-side n